

# OpenMP 与 MPI 并行计算基础

## 目 录

|                                      |           |
|--------------------------------------|-----------|
| <b>1 引言：为什么需要并行编程</b> .....          | <b>3</b>  |
| 1.1 两大并行范式 .....                     | 3         |
| 1.2 共享内存模型：UMA 与 NUMA .....          | 3         |
| <b>2 OpenMP：共享内存并行</b> .....         | <b>3</b>  |
| 2.1 什么是 OpenMP .....                 | 3         |
| 2.2 Fork-Join 执行模型 .....             | 3         |
| 2.3 第一个 OpenMP 程序：Hello OpenMP ..... | 4         |
| 2.4 指令格式与构造 .....                    | 4         |
| 2.5 工作分发构造 .....                     | 5         |
| 2.6 parallel for 循环 .....            | 5         |
| 2.7 开销：为什么不是 N 倍加速 .....             | 5         |
| 2.8 循环调度 schedule .....              | 6         |
| 2.9 嵌套循环与 collapse .....             | 6         |
| 2.10 数据冒险与作用域 .....                  | 7         |
| 2.11 三种同步方式 .....                    | 7         |
| 2.12 实战：矩阵乘法 GEMM .....              | 8         |
| 2.13 伪共享 False Sharing .....         | 9         |
| 2.14 嵌套并行区域 .....                    | 9         |
| 2.15 OpenMP 优化方法论 .....              | 9         |
| <b>3 MPI：分布式内存并行</b> .....           | <b>10</b> |
| 3.1 历史与背景 .....                      | 10        |
| 3.2 MPI 标准与实现 .....                  | 10        |
| 3.3 通信子与排名 .....                     | 10        |
| 3.4 第一个 MPI 程序 .....                 | 11        |
| 3.5 点对点通信：阻塞式发送与接收 .....             | 11        |
| 3.6 MPI_Status 与消息信封 .....           | 12        |
| 3.7 阻塞与非阻塞 .....                     | 12        |
| 3.8 通信模式 .....                       | 12        |
| 3.9 死锁问题 .....                       | 13        |
| 3.10 死锁的解决方案 .....                   | 13        |
| 3.11 非阻塞通信 .....                     | 14        |
| 3.12 同步：MPI_Test 与 MPI_Wait .....    | 14        |
| 3.13 消息顺序与公平性 .....                  | 14        |
| 3.14 集合通信 .....                      | 14        |
| 3.14.1 MPI_Barrier：同步屏障 .....        | 15        |

|                                         |           |
|-----------------------------------------|-----------|
| 3.14.2 MPI_Bcast: 一对多广播 .....           | 15        |
| 3.14.3 MPI_Scatter 与 MPI_Gather .....   | 15        |
| 3.14.4 MPI_Allgather: 全互换 .....         | 16        |
| 3.14.5 MPI_Reduce 与 MPI_Allreduce ..... | 16        |
| 3.15 实战案例: SHA512 数据验证 .....            | 17        |
| <b>4 OpenMP 与 MPI 对比与混合编程 .....</b>     | <b>18</b> |
| <b>5 本章你将学会 .....</b>                   | <b>18</b> |
| <b>6 要点速查 .....</b>                     | <b>19</b> |
| <b>7 小结 .....</b>                       | <b>19</b> |

# 1 引言：为什么需要并行编程

随着单核 CPU 的主频提升接近物理极限，摩尔定律带来的免费性能红利逐渐消失。现代处理器通过增加核心数量来提升算力，但多核硬件本身并不能自动加速串行代码。要真正利用多核，程序员必须显式编写**并行程序**（Parallel Program）。本讲介绍两种最主流的 CPU 并行编程框架：**OpenMP**（开放式多处理）和 **MPI**（消息传递接口）。

## 1.1 两大并行范式

并行编程有两大主流范式，分别对应不同的硬件架构：

- **OpenMP**：面向**共享内存**（Shared Memory）架构，在单台多核机器内实现多线程并行。所有线程共享同一地址空间，通过变量直接通信。
- **MPI**（Message Passing Interface）：面向**分布式内存**（Distributed Memory）架构，实现跨节点的多进程并行。每个进程拥有独立地址空间，通过消息传递通信。

**直觉** 不妨这样想：OpenMP 像一个团队围在同一块白板前工作，所有人都能看到白板上的内容，沟通很方便；MPI 则像分布在各地的人通过快递互寄包裹，每个人有自己的笔记本，必须显式地把信息打包发出去。

## 1.2 共享内存模型：UMA 与 NUMA

在深入 OpenMP 之前，我们了解共享内存架构的两种模型。**UMA**（Uniform Memory Access，统一内存访问）中，所有核心访问任意内存地址的延迟相同；**NUMA**（Non-Uniform Memory Access，非统一内存访问）中，内存被分区分配给不同处理器，访问本地内存比访问远端内存更快。现实中的多路服务器几乎都是 NUMA 架构，OpenMP 程序在 NUMA 机器上运行时需要注意数据局部性。

# 2 OpenMP：共享内存并行

## 2.1 什么是 OpenMP

**OpenMP**（Open Multi-Processing）是一个支持多平台共享内存多线程并行编程的 API，支持 C、C++ 和 Fortran 语言。它提供了一套编译指令（compiler directives）、运行时库函数（library routines）和环境变量（environment variables），让开发者能够方便地指定并行区域、任务和其他并行构造。

**直觉** OpenMP 的核心思想是：在已有的串行代码中插入少量编译指令，编译器就能自动生成多线程代码。你可以用最小的改动将串行程序变为并行程序，对原有代码的侵入性非常小。

## 2.2 Fork-Join 执行模型

OpenMP 采用 **Fork-Join**（派生-汇合）执行模型。程序开始时只有主线程（master thread）运行。当遇到并行区域时，主线程“派生”（fork）出一组工作线程；并行区域内，所有线程同时执行；并行区域结束后，工作线程“汇合”（join）到主线程，主线程继续串行执行。

**直觉** 想象公司里一个项目经理独自工作。遇到一个大任务时，他临时招聘一批员工 (fork)，把任务分配给大家同时干。任务完成后，员工解散 (join)，项目经理继续独自处理后续工作。

每个线程可通过 `omp_get_thread_num()` 获取自己的线程编号 (thread ID)，从 0 开始。线程总数可通过 `omp_get_num_threads()` 获取，或通过环境变量 `OMP_NUM_THREADS` 设置。

## 2.3 第一个 OpenMP 程序：Hello OpenMP

我们从一个最简单的例子开始，观察 Fork-Join 模型的实际行为：

```
#include <stdio.h>
#include <omp.h>

int main() {
    printf("Welcome to OpenMP!\n");
    #pragma omp parallel
    {
        int ID = omp_get_thread_num();
        printf("hello(%d)", ID);
        printf("world(%d)\n", ID);
    }
    printf("Bye!");
    return 0;
}
```

这段代码的执行流程：

1. `#include <omp.h>` 引入 OpenMP 头文件，声明运行时库函数。
2. `#pragma omp parallel` 是 OpenMP 指令，标记一个并行区域。编译器在此处 fork 出一组线程。
3. 并行区域内，每个线程调用 `omp_get_thread_num()` 获取编号，然后打印 hello 和 world。
4. 并行区域结束后，所有线程 join，主线程继续执行 `printf("Bye!")`。

编译时需加上 `-fopenmp` 选项，让编译器识别 OpenMP 指令：

```
gcc -o hello_omp hello_omp.c -fopenmp
```

运行时，输出中 hello 和 world 的顺序不确定，因为多个线程并发执行。但同一个线程的 hello 和 world 一定紧挨着输出，因为它们在同一线程内顺序执行。

如果不加 `-fopenmp`，编译器会忽略 `#pragma` 指令，程序按串行方式运行，输出只有一个线程的结果。

## 2.4 指令格式与构造

一条合法的 OpenMP 指令必须遵循以下格式 (C/C++)：

```
#pragma omp directive [clause[ [,] clause] ...]
```

各部分含义：

- `#pragma omp`: 固定前缀, 大小写敏感。
- `directive` (指令): 如 `parallel`、`for`、`atomic`、`critical` 等。
- `clause` (子句): 0 到多个, 用于补充指令的行为, 如 `private(x)`、`reduction(+:sum)`。

例如: `#pragma omp parallel for collapse(2) private(tmp_v, d, v)`。

指令作用于其后的代码块 (单条语句或花括号包裹的语句块)。这里需要区分两个概念:  
**指令** (`directive`) 是 `#pragma omp` 那一行; **构造** (`construct`) 则是指令加上它所作用的代码块整体。换言之, 构造是一个可执行单元, 指令只是它的起始标记。

## 2.5 工作分发构造

OpenMP 提供三种工作分发构造:

- **single**: 指定代码块只由一个线程执行 (哪个线程先到就哪个执行), 其余线程在此等待。
- **section**: 将多个独立代码块分配给不同线程并行执行。
- **for**: 将循环的迭代分配给线程组中的各线程。

其中 `for` 构造是最常用的工作分发方式, 通常与 `parallel` 组合成 `parallel for` 使用。

## 2.6 parallel for 循环

`#pragma omp parallel for` 将循环的迭代分配给各线程, 是最常见的并行化手段。以向量加法为例:

```
for (int i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```

只需在循环前添加一行指令, 编译器就会自动将迭代分配给多个线程。但要注意, 实际加速不会达到理想的  $N$  倍。

## 2.7 开销: 为什么不是 $N$ 倍加速

用 4 个线程并行执行上述向量加法, 你可能期望 4 倍加速, 但实际往往达不到。原因在于 **开销** (`Overhead`): 任何为了完成并行任务而额外消耗的计算时间、内存带宽或其他资源, 都算作开销。具体包括:

- 线程创建和销毁的开销。
- 线程同步的开销。
- 内存带宽瓶颈 (多个线程争抢同一总线)。
- 缓存一致性维护的开销。

当循环体计算量很小时, 并行化的开销可能超过并行带来的收益, 反而导致减速。

**例** 假设 4 线程并行计算  $N = 100$  的向量加法。串行耗时 100 t。如果线程创建与同步开销为 20 t，4 线程理想并行耗时  $\frac{100}{4} = 25$  t，加上开销后实际耗时  $25 + 20 = 45$  t，加速比仅  $\frac{100}{45} \approx 2.2$  倍，远低于理想的 4 倍。

## 2.8 循环调度 schedule

当循环中各迭代的计算量不同时，简单均分会导致负载不均衡：某些线程早早完成，另一些还在忙。schedule 子句控制迭代如何分配给线程：

- **static**: 静态分配，编译时均匀分块，适合负载均匀的情况。
- **dynamic**: 动态分配，线程完成一块后请求下一块，适合负载不均的情况。
- **guided**: 引导式，块大小从大到小递减，兼顾减少调度次数和负载均衡。
- **auto**: 编译器或运行时自动选择。
- **runtime**: 运行时通过环境变量决定。

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++) {
    c[i] = f(i);
}
```

```
#pragma omp parallel for schedule(dynamic, 2)
for (int i = 0; i < N; i++) {
    c[i] = f(i);
}
```

schedule(dynamic, 2) 表示每块 2 次迭代，线程完成一块后动态获取下一块。对于迭代时间差异大的循环，dynamic 调度能显著改善负载均衡，但会增加调度开销。

**例** 设  $N = 10$ ，4 个线程。static 调度下，迭代大致均分： $T_0$  得到 (0,1,2)， $T_1$  得到 (3,4,5)， $T_2$  得到 (6,7)， $T_3$  得到 (8,9)。若  $f(i)$  的耗时与  $i$  成正比 ( $f(0)$  很快， $f(9)$  很慢)，则  $T_3$  远比  $T_0$  慢。改用 schedule(dynamic, 2) 后，第一轮  $T_0$  得 (0,1)、 $T_1$  得 (2,3)、 $T_2$  得 (4,5)、 $T_3$  得 (6,7)； $T_0$  先完成，领取 (8,9)，负载更加均衡。

## 2.9 嵌套循环与 collapse

对于嵌套循环，默认情况下 parallel for 只并行化最外层。collapse(n) 子句可以将  $n$  层循环合并为一个大的迭代空间，改善负载均衡：

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        c[i][j] = a[i][j] + b[i][j];
    }
}
```

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        c[i][j] = a[i][j] + b[i][j];
    }
}
```

```
}  
}
```

合并后，OpenMP 将  $n \times n$  个迭代统一分配给各线程，比逐行分配更均衡。当某些行的计算量差异较大时，collapse 的效果尤为显著。

## 2.10 数据冒险与作用域

当多个线程同时读写同一共享变量时，会产生**数据冒险**（Data Hazard），导致结果不确定。考虑以下求和代码：

```
int a[100];  
int sum = 0;  
for (int i = 0; i < 100; i++) a[i] = i + 1;  
  
#pragma omp parallel for  
for (int i = 0; i < 100; i++) {  
    sum += a[i];  
}  
printf("Sum = %d\n", sum);
```

**直觉** 为什么 `sum += a[i]` 会有问题？因为这个操作不是原子的：它包含“读 `sum`、加 `a[i]`、写回 `sum`”三个步骤。线程 A 读到 `sum=50` 的同时，线程 B 也读到 `sum=50`，各自加上不同的值后写回，后写入的会覆盖先写入的结果，导致更新丢失。

OpenMP 提供作用域子句控制变量的可见性：

- **private**：每个线程拥有独立副本，初始值未定义。
- **shared**：所有线程共享同一变量（默认行为）。
- **firstprivate**：private 但用主线程的值初始化副本。
- **lastprivate**：private 但将最后一次迭代的值写回主线程变量。

数据冒险发生在对 shared 数据进行并发修改时，解决方法见下节。

## 2.11 三种同步方式

OpenMP 提供三种方式解决数据冒险：

1. **critical**（临界区）：基于锁，保证同一时刻只有一个线程执行临界区内的代码，可包含多条语句：

```
#pragma omp parallel for  
for (int i = 0; i < 100; i++) {  
    #pragma omp critical  
    { sum += a[i]; }  
}
```

1. **atomic**（原子操作）：利用硬件原子指令，只保护单条语句，开销比 critical 小，但仅支持有限的运算符：

```
#pragma omp parallel for  
for (int i = 0; i < 100; i++) {  
    #pragma omp atomic
```

```
    sum += a[i];
}
```

1. **reduction** (归约): 每个线程维护局部副本, 并行结束后自动归约, 性能最佳, 但仅支持有限的运算符:

```
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < 100; i++) {
    sum += a[i];
}
```

三者中, reduction 性能最好, 因为各线程独立累加局部和, 只在最后执行一次归约; atomic 次之, 利用硬件原子操作避免锁开销; critical 开销最大但最灵活, 适合保护多条语句的复合操作。

**例** 计算  $1 + 2 + \dots + 100 = 5050$ , 用 4 线程和 reduction(+:sum)。static 调度下各线程分得 25 个元素:

$$T_0: 1 + 2 + \dots + 25 = 25 \times \frac{26}{2} = 325$$

$$T_1: 26 + 27 + \dots + 50 = (26 + 50) \times \frac{25}{2} = 950$$

$$T_2: 51 + 52 + \dots + 75 = (51 + 75) \times \frac{25}{2} = 1575$$

$$T_3: 76 + 77 + \dots + 100 = (76 + 100) \times \frac{25}{2} = 2200$$

归约:  $325 + 950 + 1575 + 2200 = 5050$ , 与串行结果一致。

## 2.12 实战: 矩阵乘法 GEMM

**GEMM** (General Matrix Multiply, 通用矩阵乘法) 是科学计算的核心操作。串行版本为三重循环:

```
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        c[i][j] = 0;
        for (int k = 0; k < N; k++) {
            c[i][j] += a[i][k] * b[k][j];
        }
    }
}
```

用 OpenMP 并行化, 合并三层循环并使用 reduction:

```
#pragma omp parallel for collapse(3) reduction(+:c)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        c[i][j] = 0;
        for (int k = 0; k < N; k++) {
            c[i][j] += a[i][k] * b[k][j];
        }
    }
}
```

collapse(3) 将三重循环合并为  $N \times N \times N$  的迭代空间, reduction(+:c) 确保各线程对 c 的累加正确归约。每个线程拥有 c 的私有副本 (初始化为 0), 最终汇总。



对数组变量使用 *reduction* 需要较新的 OpenMP 版本支持。实际工程中，更常见的做法是让每个线程负责不同的行（*i* 维度），避免对 *c* 的写冲突。

## 2.13 伪共享 False Sharing

CPU 缓存以缓存行（Cache Line，通常 64 字节）为单位加载数据。如果多个线程频繁修改同一缓存行中的不同变量，缓存行会在核心之间反复失效和同步，导致性能大幅下降。这就是伪共享（False Sharing）：

```
double sum[NTHREADS];
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    for (int i = tid; i < N; i += NTHREADS)
        sum[tid] += a[i];
}
```

每个线程写 `sum[tid]`，虽然索引不同，但 `sum[0]`、`sum[1]` 等相邻元素可能在同一缓存行中。每次某个线程写入 `sum[tid]`，整个缓存行在其他核心的副本都会失效，导致大量缓存一致性流量。

解决方案包括：对数组进行对齐填充（每个元素占满一个缓存行），或直接使用 `reduction` 子句让编译器处理。

## 2.14 嵌套并行区域

OpenMP 默认禁用嵌套并行，即在并行区域内再使用 `#pragma omp parallel` 不会创建新的线程组。需要调用 `omp_set_nested` 显式启用：

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    #pragma omp parallel for
    for (int j = 0; j < n; j++) {
        c[i][j] = a[i][j] + b[i][j];
    }
}
```

嵌套并行容易导致线程数爆炸（外层  $P$  线程  $times$  内层  $Q$  线程 =  $P \times Q$  线程），通常用 `collapse` 代替嵌套并行。

## 2.15 OpenMP 优化方法论

优化一个程序时，建议遵循以下步骤：

1. **Where**（定位）：用性能分析工具（*profiling*）找到热点，找出最耗时的代码段。
2. **Why**（分析）：分析数据依赖，判断是否可以并行化。
3. **How**（方法）：选择合适的并行策略，包括任务分发、调度策略、缓存局部性、硬件环境等。
4. **Test**（测试）：实测验证，确保正确性和性能提升。

并行化的首要原则：确保正确性。在确认结果正确之前，不要急于追求性能。同时要注意开销，查阅官方文档了解细节。

## 3 MPI：分布式内存并行

### 3.1 历史与背景

在 1990 年代之前，并行计算领域有众多消息传递库，编写可移植的并行代码非常困难。1992 年的超级计算大会（Supercomputing '92）上，社区决定定义一个标准接口。1994 年，MPI-1 标准正式发布。截至 2025 年 6 月，MPI 标准已发展到 5.0 版本。

MPI 采用**消息传递模型**（Message Passing Model）：应用程序通过在进程间传递消息来完成任务，例如分配子任务、传递子问题的结果等。

### 3.2 MPI 标准与实现

MPI（Message Passing Interface）是一个标准接口规范，而非某个具体的库。存在多种实现：

- **OpenMPI**：开源实现，最常用。
- **MPICH**：另一个开源实现。
- **Intel-MPI**：Intel 提供，包含在 Intel oneAPI 中。
- **HMPI**（Hyper-MPI）：华为提供的实现。

请注意区分 MPI 标准（规范文档）和 MPI 实现（具体的库）。不同实现都遵循同一标准，代码可以在不同实现间移植。

### 3.3 通信子与排名

MPI 的核心概念是**通信子**（Communicator），它定义了一组可以互相通信的进程。每个进程在通信子内有一个唯一的**排名**（rank），从 0 开始编号。MPI\_COMM\_WORLD 是默认的全局通信子，包含所有进程。

**直觉** 通信子就像一个“群聊”。在群里每个人都有有一个编号（rank），MPI\_COMM\_WORLD 是“全员群”。你还可以通过 MPI\_Comm\_split 把大群拆成若干小群，实现分组通信。

MPI\_Comm\_split 可以将通信子拆分为子组。参数包括：comm（基础通信子）、color（决定进程属于哪个新组）、key（组内排名顺序）、new\_comm（输出的新通信子）。

通过 MPI\_Comm\_rank 获取当前进程的 rank，通过 MPI\_Comm\_size 获取通信子内进程总数：

```
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

rank 标识进程身份，size 表示进程总数。通过 rank 判断，不同进程可以执行不同的代码路径，实现任务分工。

### 3.4 第一个 MPI 程序

一个最小的 MPI 程序：

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int name_len;
    MPI_Get_processor_name(processor_name, &name_len);

    printf("Hello world from processor %s, rank %d out of %d processors\n",
           processor_name, world_rank, world_size);

    MPI_Finalize();
    return 0;
}
```

每个 MPI 程序必须以 MPI\_Init 开始、MPI\_Finalize 结束。各部分说明：

1. MPI\_Init(&argc, &argv)：初始化 MPI 环境。
2. MPI\_Comm\_size / MPI\_Comm\_rank：获取进程总数和当前进程编号。
3. MPI\_Get\_processor\_name：获取处理器名称。
4. MPI\_Finalize()：清理 MPI 环境，之后不能再调用 MPI 函数。

编译和运行：

```
mpicc -o hello_mpi hello_mpi.c
mpirun -np 4 ./hello_mpi
```

mpicc 是 MPI 的编译 wrapper，-np 4 指定启动 4 个进程。在集群上运行时，通常通过 Slurm 提交作业，由调度器分配节点和进程。

### 3.5 点对点通信：阻塞式发送与接收

MPI\_Send 和 MPI\_Recv 是最基本的点对点通信函数：

```
int MPI_Send(const void* buffer, int count,
             MPI_Datatype datatype, int recipient,
             int tag, MPI_Comm communicator);

int MPI_Recv(void* buffer, int count,
             MPI_Datatype datatype, int sender,
             int tag, MPI_Comm communicator,
             MPI_Status* status);
```

参数说明：

- **buffer**: 发送/接收缓冲区。
- **count**: 元素个数。
- **datatype**: 元素的数据类型 (如 `MPI_INT`、`MPI_DOUBLE`)。
- **recipient / sender**: 目标/来源进程的 rank。
- **tag**: 消息标签, 用于区分不同消息。
- **communicator**: 通信子。
- **status (仅 Recv)**: 返回接收状态信息。

基本用法:

```
if (rank == 0) {
    MPI_Send(data, n, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD);
} else if (rank == 1) {
    MPI_Recv(data, n, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, &status);
}
```

**阻塞 (Blocking)** 意味着 `MPI_Send` 在消息可以被安全覆盖后返回, `MPI_Recv` 在数据到达后才返回。

### 3.6 MPI\_Status 与消息信封

`MPI_Recv` 的 `status` 参数是一个 `MPI_Status` 结构体, 至少包含三个属性:

- **MPI\_SOURCE**: 消息的实际发送方 rank。
- **MPI\_TAG**: 消息的实际标签。
- **MPI\_ERROR**: 错误码。

如果不需要状态信息, 可以传入 `MPI_STATUS_IGNORE`。当接收方使用 `MPI_ANY_SOURCE` 或 `MPI_ANY_TAG` 时, 可以通过 `status` 查询实际的发送方和标签。

消息除了数据部分, 还携带**信封 (Envelope)** 信息用于区分和选择性接收, 包括: `source` (发送方)、`destination` (接收方)、`tag` (标签) 和 `communicator` (通信子)。

### 3.7 阻塞与非阻塞

MPI 通信分为阻塞和非阻塞两种模式:

- **阻塞 (Blocking)**: 函数在消息数据和安全存储完成前不返回。消息可能直接复制到匹配的接收缓冲区, 也可能先复制到系统临时缓冲区。
- **非阻塞 (Non-blocking)**: 函数发起操作后立即返回, 不等待完成。通过 `MPI_Request` 对象跟踪操作状态, 稍后用 `MPI_Wait` 或 `MPI_Test` 检查完成情况。

**直觉** 阻塞通信像打电话: 你必须等到对方接听才能继续。非阻塞通信像发短信: 发出去就可以去做别的事, 稍后再查看是否收到回复。

### 3.8 通信模式

MPI 的阻塞发送有四种通信模式, 区别在于开始和完成的条件:

| 通信模式 | 开始时间 | 完成时间 |
|------|------|------|
|------|------|------|

|                  |              |              |
|------------------|--------------|--------------|
| Buffer (缓冲)      | 立即           | 消息已进入系统缓冲区   |
| Synchronous (同步) | 立即           | 匹配的 Recv 已发布 |
| Ready (就绪)       | 匹配的 Recv 已发布 | 发送缓冲区可重用     |
| Standard (标准)    | 取决于实现        | 取决于实现        |

对应的函数名分别为 MPI\_Bsend、MPI\_Ssend、MPI\_Rsend 和 MPI\_Send (标准模式)。其中同步模式 MPI\_Ssend 总是等待接收方发布匹配的 Recv 后才完成，这常常是死锁的根源。

### 3.9 死锁问题

当两个进程互相等待对方接收时，就会发生**死锁** (Deadlock)。以下代码用 MPI\_Ssend 交换数据：

```
int my_rank;
MPI_Comm_rank(comm, &my_rank);
MPI_Ssend(sendbuf, count, MPI_INT, my_rank ^ 1, tag, comm);
MPI_Recv(recvbuf, count, MPI_INT, my_rank ^ 1, tag, comm, &status);
```

$my\_rank \oplus 1$  通过异或运算得到对方 rank ( $0 \oplus 1 = 1$ ,  $1 \oplus 1 = 0$ )。两个进程都先执行 MPI\_Ssend，而同步发送要求对方先发布 MPI\_Recv 才能完成。双方都卡在 Send，谁也到达不了 Recv，于是永远阻塞。

**直觉** 两个人面对面站着，都想把自己的东西递给对方，但都要求对方先伸手接住。于是两人都举着东西僵持不动，这就是死锁。

### 3.10 死锁的解决方案

解决死锁有多种方法。

**方法一：调整顺序。**让一个进程先接收，另一个先发送：

```
if (my_rank == 0) {
    MPI_Ssend(sendbuf, count, MPI_INT, 1, tag, comm);
    MPI_Recv(recvbuf, count, MPI_INT, 1, tag, comm, &status);
} else if (my_rank == 1) {
    MPI_Recv(recvbuf, count, MPI_INT, 0, tag, comm, &status);
    MPI_Ssend(sendbuf, count, MPI_INT, 0, tag, comm);
}
```

rank 0 先发，rank 1 先收，完成后 rank 1 再发、rank 0 再收，不会死锁。

**方法二：使用 MPI\_Sendrecv。**MPI\_Sendrecv 在一次调用中同时完成发送和接收，由 MPI 内部协调避免死锁：

```
MPI_Sendrecv(sendbuf, count_send, MPI_INT, recipient, tag_send,
             recvbuf, count_recv, MPI_INT, sender, tag_recv,
             MPI_COMM_WORLD, &status);
```

MPI\_Sendrecv 的发送缓冲区和接收缓冲区必须是不同的内存区域。

**方法三：使用非阻塞通信。**将发送或接收改为非阻塞，允许通信进行时继续执行后续代码。

### 3.11 非阻塞通信

非阻塞通信的函数名以 I 开头 (Immediate)，返回一个 MPI\_Request 对象，允许在通信进行时继续计算，实现**计算与通信重叠**：

```
MPI_Request request;
MPI_Isend(data, n, MPI_DOUBLE, dst, tag, MPI_COMM_WORLD, &request);
do_computation();
MPI_Wait(&request, &status);
```

MPI\_Isend 发起发送后立即返回，程序继续执行 do\_computation()。完成后调用 MPI\_Wait 阻塞等待通信完成。MPI\_Irecv 的用法类似。

用非阻塞通信也可以解决前面的死锁问题：

```
MPI_Request req;
MPI_Isend(sendbuf, count, MPI_INT, my_rank ^ 1, 0,
          MPI_COMM_WORLD, &req);
MPI_Recv(recvbuf, count, MPI_INT, my_rank ^ 1, 0,
          MPI_COMM_WORLD, MPI_STATUS_IGNORE);
MPI_Wait(&req, MPI_STATUS_IGNORE);
```

MPI\_Isend 立即返回，程序继续执行 MPI\_Recv，两个进程都能到达接收，死锁解除。

### 3.12 同步：MPI\_Test 与 MPI\_Wait

对于非阻塞操作，MPI 提供两种检查完成的方式：

- MPI\_Wait(request, status)：阻塞等待，直到对应非阻塞操作完成。
- MPI\_Test(request, &flag, status)：非阻塞检查，flag 为 true 表示已完成，为 false 表示未完成。

MPI\_Waitall 可以同时等待多个请求完成。非阻塞通信配合 MPI\_Test 可以实现轮询式的工作模式：在等待通信完成的同时执行其他计算。

### 3.13 消息顺序与公平性

**消息不超车 (Non-overtaking)** 规则：来自同一发送方、发往同一接收方、具有相同标签的消息，按发送顺序匹配接收。但注意，此规则仅在单线程环境下保证。不同发送方或不同标签的消息不保证到达顺序。

**公平性 (Fairness)**：MPI 不保证公平性。如果多个进程同时向同一接收方发送消息，某些发送方可能一直得不到匹配，产生**饥饿 (Starvation)**。例如，rank 1 和 rank 2 同时向 rank 0 发送消息，rank 0 用 MPI\_ANY\_SOURCE 接收，MPI 不保证 rank 1 和 rank 2 被接收的概率均等。

### 3.14 集合通信

**集合通信 (Collective Communication)** 涉及通信子内所有进程，必须由所有进程共同调用。与线性逐个点对点通信 ( $O(n)$  复杂度) 相比，集合通信通常采用树形实现 ( $O(\log n)$  复杂度)，在大规模集群中优势明显。

### 3.14.1 MPI\_Barrier: 同步屏障

MPI\_Barrier 让通信子内所有进程在此处等待，直到全部到达后才继续：

```
MPI_Barrier(MPI_COMM_WORLD);
```

屏障会影响性能，应谨慎使用。在不需要严格同步的场景中，应避免不必要的屏障调用。

### 3.14.2 MPI\_Bcast: 一对多广播

MPI\_Bcast 将根进程的数据广播给通信子内所有进程：

```
MPI_Bcast(buffer, count, MPI_DATATYPE, root, MPI_COMM_WORLD);
```

为什么不直接用 MPI\_Send / MPI\_Recv 逐个发送？因为 Bcast 采用**树形算法**（Tree-based Algorithm），通信复杂度从线性的  $O(n)$  降为  $O(\log n)$ 。

**例** 设 8 个进程，root=0。线性方式：root 依次发给 1, 2, ..., 7，需要 7 步。树形方式：  
第 1 步：0 → 4  
第 2 步：0 → 2, 4 → 6  
第 3 步：0 → 1, 2 → 3, 4 → 5, 6 → 7  
仅需  $\log_2 8 = 3$  步。当进程数更大时，差距更明显。

用 MPI\_Wtime() 计时可以对比两种方式的性能差异：

```
double start = MPI_Wtime();
if (my_rank == 0) {
    for (int i = 1; i <= 31; i++)
        MPI_Send(sendbuf, 0x10000, MPI_INT, i, 0, MPI_COMM_WORLD);
} else {
    MPI_Recv(recvbuf, 0x10000, MPI_INT, 0, 0, MPI_COMM_WORLD,
             MPI_STATUS_IGNORE);
}
double end = MPI_Wtime();

start = MPI_Wtime();
MPI_Bcast(&sendbuf, 0x10000, MPI_INT, 0, MPI_COMM_WORLD);
end = MPI_Wtime();
```

在大规模进程数下，Bcast 比逐个 Send/Recv 快得多。

### 3.14.3 MPI\_Scatter 与 MPI\_Gather

**Scatter**（分散）将根进程的数据分块发送给各进程，每进程收到不同的一块。**Gather**（收集）则相反，将各进程的数据收集到根进程。

```
MPI_Scatter(sendbuf, count_send, MPI_DATATYPE,
            recvbuf, count_recv, MPI_DATATYPE,
            root, MPI_COMM_WORLD);
```

```
MPI_Gather(sendbuf, count_send, MPI_DATATYPE,
            recvbuf, count_recv, MPI_DATATYPE,
            root, MPI_COMM_WORLD);
```



Scatter 和 Gather 常配合使用：先 Scatter 分发数据，各进程计算局部结果，再 Gather 汇总。以计算平均值为例：

```
MPI_Scatter(buffer, n/4, MPI_DOUBLE,
            local_buffer, n/4, MPI_DOUBLE,
            0, MPI_COMM_WORLD);
```

```
double local_avg = 0;
for (int i = 0; i < n/4; i++) {
    local_avg += local_buffer[i];
}
local_avg /= n/4;
```

```
double avgs[4];
MPI_Gather(&local_avg, 1, MPI_DOUBLE,
          avgs, 1, MPI_DOUBLE,
          0, MPI_COMM_WORLD);
```

根进程将数组分成 4 份分发给各进程，每个进程计算局部平均值，再汇总到根进程的 avgs 数组。

**例** 设 4 个进程，数据为 (2, 4, 6, 8)，每个进程分得一个元素。各进程的 local\_avg 就是自己的值： $P_0$  得 2， $P_1$  得 4， $P_2$  得 6， $P_3$  得 8。Gather 后根进程得到 avgs = (2, 4, 6, 8)，全局平均 =  $\frac{2+4+6+8}{4} = 5.0$ 。

### 3.14.4 MPI\_Allgather: 全互换

MPI\_Allgather 等价于先 Gather 再 Bcast：每个进程都获得所有进程的数据。换言之，Gather 的结果不再只存在根进程，而是所有进程都有一份完整的副本。

### 3.14.5 MPI\_Reduce 与 MPI\_Allreduce

MPI\_Reduce 对各进程的数据进行归约操作（求和、求最大值等），结果存放在根进程：

```
MPI_Reduce(&local_val, &global_val, 1, MPI_DOUBLE,
           MPI_SUM, 0, MPI_COMM_WORLD);
```

用 Reduce 可以更简洁地实现前面的平均值计算：

```
double local_avg = compute_local_avg(local_buffer);
double global_avg;
MPI_Reduce(&local_avg, &global_avg, 1, MPI_DOUBLE,
           MPI_SUM, 0, MPI_COMM_WORLD);
if (rank == 0) {
    global_avg /= size;
    printf("Average: %f\n", global_avg);
}
```

MPI\_Allreduce 在归约后将结果发送给所有进程，常用于需要所有进程都知道全局结果的场景，如迭代法中的残差检查。



**例** 接前例，4 个进程的 local\_avg 分别为 2.0, 4.0, 6.0, 8.0。MPI\_Reduce(MPI\_SUM) 后根进程得到  $2 + 4 + 6 + 8 = 20$ ，除以 4 得全局平均 5.0。如果改用 MPI\_Allreduce，则所有 4 个进程都得到 20，各自可以独立计算平均值。

### 3.15 实战案例：SHA512 数据验证

本例来自 HPC Game 2024，展示了如何利用非阻塞 MPI 实现 I/O 与计算的重叠。

**任务：**实现基于 SHA512 的数据验证算法。步骤为：

1. 将输入文件切分为 1MB 的块（最后一块不足 1MB 则补零）。
2. 第  $i$  块的验证和 = SHA512(第  $i$  块数据 || 第  $i-1$  块的验证和)。
3. 最后一块的验证和即为整个文件的验证和。

**难点分析：**第  $i$  块的计算依赖第  $i-1$  块的结果，计算本身是串行的，无法直接并行化。但文件 I/O 是独立的，可以与计算重叠。

**关键思路：**将数据块分配给多个进程，每个进程负责一段连续的块。进程间通过非阻塞通信传递上一块的验证和。在等待接收验证和的同时，进程可以执行文件读取和部分摘要计算，实现**计算与通信重叠**。

核心代码（简化版）：

```
for (int i = start_block; i < upper_bound; i++) {
    if (i != 0) {
        MPI_Irecv(prev_md, SHA512_DIGEST_LENGTH, MPI_UINT8_T,
                  sender, 0, MPI_COMM_WORLD, &request);
    }
    istrm.seekg(i * BLOCK_SIZE);
    istrm.read(reinterpret_cast<char*>(data + i * BLOCK_SIZE),
               std::min(BLOCK_SIZE, file_size - i * BLOCK_SIZE));

    for (int j = i; j < upper_bound; j++) {
        uint8_t buffer2[BLOCK_SIZE]{};
        EVP_DigestInit_ex(ctx[j-i], sha512, nullptr);
        std::memcpy(buffer2, data + j * BLOCK_SIZE,
                     std::min(BLOCK_SIZE, len - j * BLOCK_SIZE));
        EVP_DigestUpdate(ctx[j-i], buffer2, BLOCK_SIZE);
    }

    if (i != 0) {
        MPI_Wait(&request, MPI_STATUS_IGNORE);
    }

    for (int j = i; j < upper_bound; j++) {
        EVP_DigestUpdate(ctx[j-i], prev_md, SHA512_DIGEST_LENGTH);
        EVP_DigestFinal_ex(ctx[j-i], prev_md, &len);
    }

    if (upper_bound != num_block) {
        MPI_Isend(prev_md, SHA512_DIGEST_LENGTH, MPI_UINT8_T,
```

```

        receipient, 0, MPI_COMM_WORLD, &request);
    }
}

```

代码流程：

1. MPI\_Irecv 非阻塞接收上一进程传来的上一块验证和。
2. 在等待期间，执行文件读取 (istrm.read) 和部分摘要计算 (EVP\_DigestUpdate)。
3. MPI\_Wait 确保验证和已到达后，将其混入摘要计算 (第二个 EVP\_DigestUpdate)。
4. EVP\_DigestFinal\_ex 完成当前块的摘要。
5. MPI\_Isend 非阻塞发送当前块的验证和给下一进程。

*EVP\_DigestUpdate(a); EVP\_DigestUpdate(b);* 等价于 *EVP\_DigestUpdate(concat(a, b))*，因此可以将块数据和前一块验证和分两次 *Update*。

**直觉** 这个案例的精妙之处在于：虽然计算是串行依赖的，但文件 I/O 可以提前进行。通过非阻塞通信，进程在等待前一块验证和的同时，先把后续块的数据从磁盘读入内存并做部分计算。这样，当验证和到达时，大部分工作已经完成，只需最后一步混合即可。

## 4 OpenMP 与 MPI 对比与混合编程

两种并行范式的对比如下：

| 特性   | OpenMP      | MPI          |
|------|-------------|--------------|
| 内存模型 | 共享内存        | 分布式内存        |
| 并行粒度 | 线程          | 进程           |
| 通信方式 | 共享变量        | 消息传递         |
| 适用范围 | 单节点多核       | 跨节点集群        |
| 编程难度 | 较低          | 较高           |
| 可扩展性 | 受限于核心数      | 可扩展到上万核      |
| 启动方式 | 编译指令 + 环境变量 | mpirun -np N |

在实际 HPC 应用中，常采用**混合编程** (Hybrid Programming)：MPI 负责跨节点通信，OpenMP 在节点内利用多核。这样每个 MPI 进程管理一个节点，节点内通过 OpenMP 实现多线程并行，既减少了 MPI 通信量，又充分利用了共享内存的效率。这种 MPI + OpenMP 的混合模式是当前超算应用的主流编程范式。

## 5 本章你将学会

1. 理解共享内存与分布式内存两种并行架构的区别，以及 UMA 与 NUMA 的概念。
2. 使用 OpenMP 的 parallel、parallel for、collapse、schedule 等指令编写多线程并行程序。
3. 识别和解决数据冒险，掌握 critical、atomic、reduction 三种同步方式的适用场景。
4. 理解 MPI 的通信子、rank、阻塞与非阻塞通信的概念，编写基本的点对点 and 集合通信程序。

5. 分析死锁的成因并运用重排序、MPI\_Sendrecv 和非阻塞通信解决死锁。

## 6 要点速查

OpenMP 指令速查：

| 指令/子句                    | 功能       | 备注                         |
|--------------------------|----------|----------------------------|
| #pragma omp parallel     | 创建并行区域   | Fork-Join 模型               |
| #pragma omp parallel for | 并行化循环    | 循环必须可并行化                   |
| collapse(n)              | 合并 n 层循环 | 改善负载均衡                     |
| schedule(kind, chunk)    | 设定调度策略   | static/dynamic/guided/auto |
| private(var)             | 每线程独立副本  | 初始值未定义                     |
| reduction(op:var)        | 归约操作     | 性能最佳                       |
| #pragma omp critical     | 临界区      | 基于锁                        |
| #pragma omp atomic       | 原子操作     | 单条语句                       |

MPI 函数速查：

| 函数                            | 功能          | 类型  |
|-------------------------------|-------------|-----|
| MPI_Init / MPI_Finalize       | 初始化/清理      | 必须  |
| MPI_Comm_rank / MPI_Comm_size | 获取 rank/进程数 | 查询  |
| MPI_Send / MPI_Recv           | 阻塞式点对点通信    | 点对点 |
| MPI_Isend / MPI_Irecv         | 非阻塞式点对点通信   | 点对点 |
| MPI_Wait / MPI_Test           | 等待/检查非阻塞操作  | 同步  |
| MPI_Sendrecv                  | 同时发送和接收     | 点对点 |
| MPI_Barrier                   | 同步屏障        | 集合  |
| MPI_Bcast                     | 一对多广播       | 集合  |
| MPI_Scatter / MPI_Gather      | 分发/收集       | 集合  |
| MPI_Allgather                 | 全互换         | 集合  |
| MPI_Reduce / MPI_Allreduce    | 归约/全归约      | 集合  |

## 7 小结

本讲介绍了 CPU 并行编程的两大基石。OpenMP 通过简单的编译指令实现共享内存多线程并行，适合单节点内的快速并行化，但需要注意数据冒险、伪共享和开销等问题。MPI 通过消息传递实现分布式内存多进程并行，可扩展到大规模集群，但编程复杂度更高，需要处理通信、死锁和负载均衡等问题。两者并非互斥，混合编程是超算应用的主流范式。

下一讲将进入 GPU 并行编程，学习如何利用 GPU 的大规模并行能力加速计算密集型任务。